

看完某研发部总监的缺陷分析报告后，我问：“你们什么时候做分析？”

总监：“产品发版后。”他们最近一次发版是八月份，立项早在三月份已经开始。

我：“如果你们分析的目的是之后做改进，发版后才分析的作用不大。要想有真正的效果，就必须让开发团队每次 2-4 周迭代后一起做回顾分析。”

虽然总监说很赞同，但估计他心里想：“我们这几个月一直天天都很忙，中间怎么能腾出时间做回顾复盘！”

一条原则应能改变他的思路，从 2013 年起，这原则一直驱动我每天每月做规划。

2017 年

我有一位拥有 30 多年电机工程经验的老同学李先生，他对 Linux 感兴趣并持续研究，这几年又开始玩树莓派，对此也非常精通。

我找到他，希望在树莓派上安装 Ipython、Jupyter Notebook 等开源应用软件。一直以来，李先生的理念就是尽量用最简单、最轻易的方法去解决客户的问题，不浪费资源。2017 年，我开始尝试建立 WIKI 服务器给客户使用，他推荐我用树莓派。

他说：“树莓派不仅省电而且快，也不知道你这个概念是否持久，我不会浪费时间——在大型服务器上安装 Linux + Mediawiki，如果你觉得这（树莓派）可以便继续用。如果觉得它性能不够，下一步我帮你换更大型的服务器。”

一直到现在，已经超过 8 年了，我们一直还是继续用树莓派来协助评估（树莓派从原来的 Pi2(1G 内存)，现在已经是 P4(4G 内存)，服务的客户也超过了百家。这证明他的思路是对的。

还有一个细节，当时他安装好 Jupyter Notebook 程序包后，一直没有后续工作。我问他什么原因？

他说：“很简单，我不熟悉软件工程，也不懂如何测试。肯定要找测试一下，我才知道有没有做错。这样可以避免无谓的返工。”所以我们就约好周日我回到办公室跟他远程测试。

李先生打开树莓派后，我用 VNC 客户端进入他的树莓派，打一些最基本的 Ipython 指令，验证没问题。

下一步我要跑一些比较复杂的、要用到 Pandas 包的 Python 程序，却发现还没有安装 Pandas 包。他马上到网上去找安装介质以及树莓派如何安装 Pandas 的方法。处理好后，我再跑对应的程序，包括之前的指令全部跑通。前后花了 2 小时，测试工作按原计划成功。

资源有限、用最低投入达到客户要求，不多做。每当李先生做完一步，如果没有通过测试，就不会浪费时间走下一步。

每小步验证

1983 年，本科最后一年我做毕业项目。导师建议我们参考一些常用的语言发声算法，尝试在专门做信号处理的芯片上写程序，读出一些文字或句子（与现在不同，当时这项技术还没有成熟，很多大学还在研究），虽然我预计有不小的技术难度，但觉得这很先进，便与另一位同学合作，开始制订项目计划。

我们全力投入，一边研究语言发声的算法，一边并行设计电子线路和软件等。我们用了 2~3 个月的时间做了整个电子线路板的硬件，同时还设计了整个软件架构，并使用计算机模拟，验证每个字实现算法后的发声效果。因为最后一年课程很多，时间过得很快，从 9 月开始用了 6 个月时间，软硬件的设计与开发

终于都完成了。但是最终没有发出声音，更不要说能读出一些字和句子了，项目以失败告终。

还有一次失败经历让我印象深刻。在大学的第三年我有幸进入大东电报局 (Cable & Wireless) 电报业务运维部门实习，该部门当时在开发一套新的电脑系统以取代本来基于 UNIVAC 的电报系统，总工程师让我用半年时间，在一个微机上编程从那些电脑上收集重要信息，如果发现异常就警报。按照惯例我前 3 个月花精力做整个系统设计，还买了一些展示电子版，准备用来展示，但由于经验不足，半年后最终什么都没有展示出来。

到了 2000 年，在我兼读软件工程硕士课程时，开始接触到敏捷开发，才意识到两次项目失败的主因：不应该花大量时间去追求前期完美的设计，而是应该逐步迭代，先做一些最基本的简单功能，每小步验证，逐步优化。例如，我的毕业项目中应该先做出最基本的硬件、软件，起码能够发出声音，因为整个项目是开创性的，从未有过成功的实验。

敏捷大师 Dave Thomas 先生在 2015 年演讲里提出敏捷软件开发的核心是：

1. 向你的目标迈出一小步
2. 从反馈调整你的理解
3. 重复
 - 当两种或以上选择的价值大致相同时，选一条让未来更容易修改（软件）的路径。

这些经历让我体会到为什么我们在写程序时，必须先想一下该怎么测试。每小步验证算是敏捷原则里“要频繁地交付”，但其实是精益（LEAN）的核心思路。

精益？是敏捷的概念吗？为什么 2001 年提出的敏捷宣言和 12 条原则都没有提精益？

很多敏捷团队使用“看板 (Kanban)”，其实并非源自宣言或 12 条原则，而是来自丰田的 Just-in-time——按客户订单拉动物料的采购。

Mary Poppendieck 具有多年编码经验，也为 3M 公司开发系统，完善生产流程，很了解精益的力量，2003-2010 年间出版了 3 本著作，提出软件开发也可以使用精益做改善（详见参考）。

精益强调：

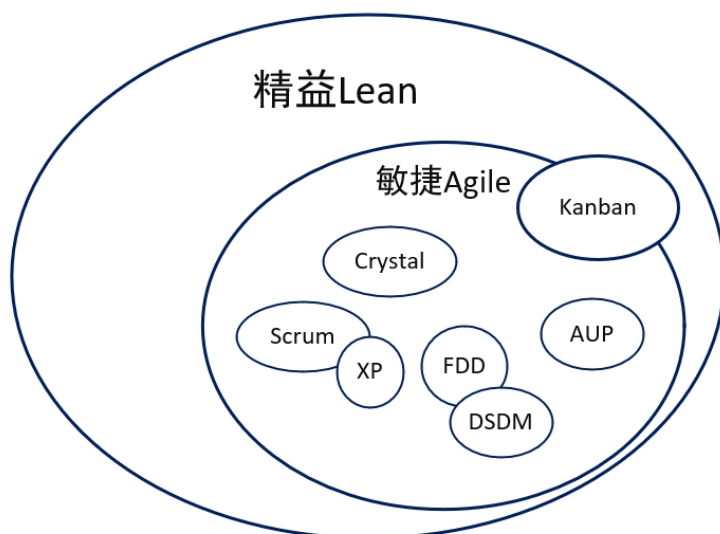
1. 减少浪费
2. 确保质量、零缺陷
3. 快速交付
4. 不要过早下定论，尽量延后承诺

虽然策划重要，但不是死板地按计划执行，制定几个月的长期计划，要灵活按迭代交付后再优化策划。第二章里谷歌创始人 Larry Page 拒收 CEO Eric Schmidt 的商业计划书“芬兰计划”，请他先与谷歌工程师聊聊。这和包括频繁交付 (Ship and iterate)、管好失败 Fail well、针对客户 (Focus on the user)

等谷歌文化，都属于精益。

丰田专注零缺陷、零库存，都是精益。

精益与敏捷是相对吗？敏捷宣言和 12 条原则主要是针对当时软件开发的各种不良现象；精益软件开发更强调整个系统的持续改善。两者是互补的，相互渗透，敏捷与精益之间的区分也逐渐模糊。所以敏捷开发演化至今已经包含了精益思路。PMI 的 ACP 把敏捷归入精益其中：



精益可以用于软件开发吗？完全可以但需要变通地理解。可以先从了解软件开发浪费开始。

浪费 (Waste)

在软件开发，除了缺陷外还有哪些浪费？除了软件开发的缺陷（对应生产的不良品）外，其他 6 类生产浪费都能找到对应：

工业生产	软件开发
过程中库存 (In-Process Inventory)	半成品 (Partially Done Work)
动作 (Motion)	任务切换 (Task Switching, Multi-tasking)
生产过多 (Over Production)	多余的功能 (Extra Features)
加工 (Extra Processing)	重新学习 (Relearning)
搬运 (Transportation)	交接 (Handovers)
等待 (Waiting)	延误 (Delay)

除了缺陷，和多余的功能（会在后面软件需求里讨论），其他的浪费的说明如下：

半成品

未完成的产出物包括：

1. 没有对应软件代码的文档
2. 没有测试过的代码
3. 没有文档说明的代码
4. 代码没有实际安装使用

任务替换

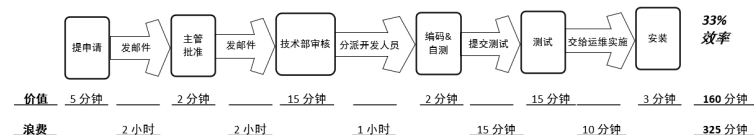
原因可能是要同时处理几个开发任务，导致无法集中完成任务；过后想完成时要多花时间，引起返工。

注意：请不要误以为不应同时做多个项目。例如，写文章或编程，如果没有思路无法进展，我会转做其他；生产率超高的作曲家舒伯特同时会创作多首作品。所以前提是策划任务时，要让每项任务有足够的时间“完成”。

延误 (Delay)

在软件维护，引起延误的最大原因是什么？通常是系统或流程的问题，并非开发人员的质量水平。

精益里有价值流 (Value Stream) 分析，可以区分哪些过程是产生价值，哪些是浪费。



例如：从客户提出需求到最终完成修正、使用，前后共需要 8 小时，但对客户有价值的过程只占 33%（2.5 小时）。

如果能减少或省略无价值的过程，如发电邮到主管和组长等待审批，就能减少延误。

软件交接 (Handovers)

因为接受方不能完全理解，最终产生返工，导致浪费。例如开发完后交给其他人维护，如果他之前没有参与，便难以顺利交接。

重新学习 (Relearning)

开发是不断学习、创造的过程，因没有及时做记录，过后很多时候自己也忘记了，导致后面重复，引起浪费。

第二章里，Weisbord 先生印刷公司的订单处理部门便是一例：，在开始团队制前，任务都是按业务流程分工，会产生以下浪费：

任务交接 - 从输入订单，检查客户信用，发起内部生产订单等，各由专人负责，导致很多任务交接或任务替换。

半成品 - 从收到客户订单，到交付产品后，收到支票并内部对账，过程中的产出物需要等待下游处理。

后面变成团队负责制，团队自己处理所有相关任务便能减少这些浪费，也能减少整个订单的处理时间。

软件开发公司也类似，例如：

专门开发游戏的公司，为了加快开发出新游戏的速度也会从原本按功能部门分工改成团队负责制，每个团队负责一款游戏，除了能更快速开发出新游戏外，也能减少浪费。

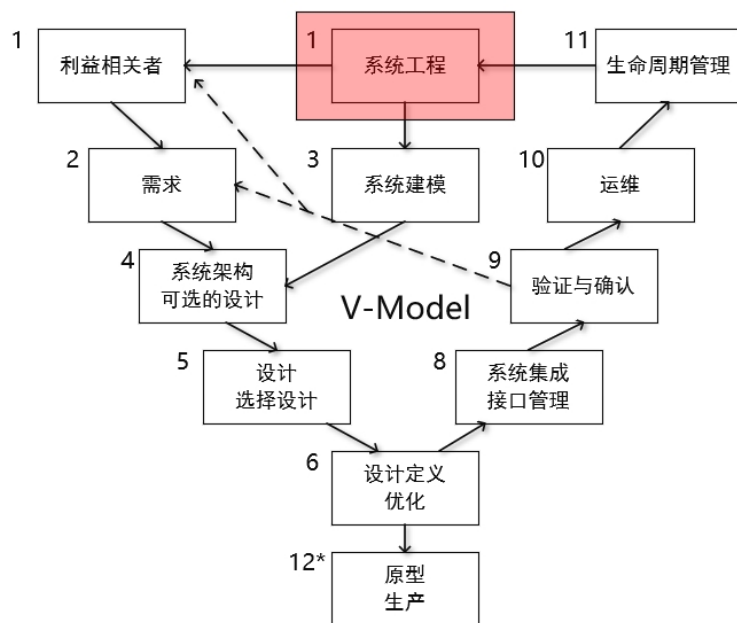
非开发人员如何应用精益

大家都想尽量避免半完成的任务，避免浪费。所以精益的应用范围很广。随便在网上搜索“精益”，便能看到“精益管理”、“精益思想”、“精益创业”等各类应用。例如，如果今周每天都有培训安排，只有晚上能腾出 2 小时左右完善这本书，我会把时间用于哪些改动不大的章节，因从历史数据，如果某章就须要大规模重写，会起码要 1 - 2 天才能完成，会尽量安排在周末做，尽量避免在零散的时间段里做，减少浪费。

例如金庸写长篇武侠小说都是每周在香港《明报》发布，最后累加成书。因每周每天可以腾出的时间有限，我自己也是使用精益思路写书：写完一章后先自己读一篇，修正错误；然后发到群里让朋友圈读，按反馈修正；尽量定期在 CSDN 发布，每 2 周发下一章，给自己压力尽早完成，但因之前已经印刷了 2 版，发现出现太多应可避免的错误，所以也不断提醒自己不能因进度而牺牲质量。

系统工程 V 模型

从 50 年代开始出现编程语言，软件程序越来越庞大，越来越复杂，难以维护。是否可以用系统工程思路来改善呢？系统工程主要用于比如飞机、汽车等复杂系统，要设计新款飞机，先从需求开始，按 V 模型，验证确认后才可以最后交付。例如按系统工程的概念，我一个产品例如汽车，应该先有个产品分解的思路，汽车由车轮、引擎、油缸或者电池等各种组件组成，应先有总体设计，然后一层一层分下去，里面也可能包括一些软件组件。



虽然除了缺陷外，其它 6 类浪费也会有极大影响，但因这部分主要讨论软件工程，下章会从缺陷的角度，看看导致软件难以维护的原因与改善措施。

本章最佳实践对应：

- PMI、ACP
 - 里面持续改善 (Continuous Improvement)，价值驱动交付 (Value Driven Delivery)，策划 (Adaptive Planning) 等领域都能对上精益思路
- CMMI
 - 虽然没有特定实践域包含持续改善，但 PCM、CAR、MPM 等实践域都支持精益的持续改善

参考 References

1. Poppendieck, Mary, Poppendieck, Tom *Lean Software Development* (2003) Pearson Education
2. Poppendieck, Mary, Poppendieck, Tom *Implementing Lean Software Development: from concept to cash* (2007) Pearson Education
3. Poppendieck, Mary, Poppendieck, Tom *Leading Lean Software Development* (2010) Pearson Education
4. *Agile Practice Guide* Project Management Institute(PMI) (2017)